

Assistente vocale con logica di dominio esterna

L'HUD come linguaggio, gli eventi come algebra: architettura teorica, mappa del sistema attuale, cosa cambierebbe davvero

Progetto MiniCPM-o-Demo (ramo HUD) · 05/09/2026 · sintesi di tre giorni di misure (plan/ramo-hud.md, analisi-toolcalling.md)

1. Il problema in una riga

Un modello vocale end-to-end (MiniCPM-o 4.5: ascolta e parla in full-duplex, 1 unità al secondo) non emette chiamate di funzione e non ha un ingresso testuale in tempo reale. L'unico canale laterale che ha è la **visione**: un fotogramma per unità, 64 token. La logica di dominio (prenotazioni, DB, regole) deve quindi vivere **fuori** dal modello, e comunicare con lui attraverso un'immagine. Il resto dell'architettura discende da questo vincolo.

2. L'algebra

Tre oggetti e tre funzioni. Tutto ciò che è di dominio sta in un solo posto (la macchina δ); il modello che parla non conosce regole.

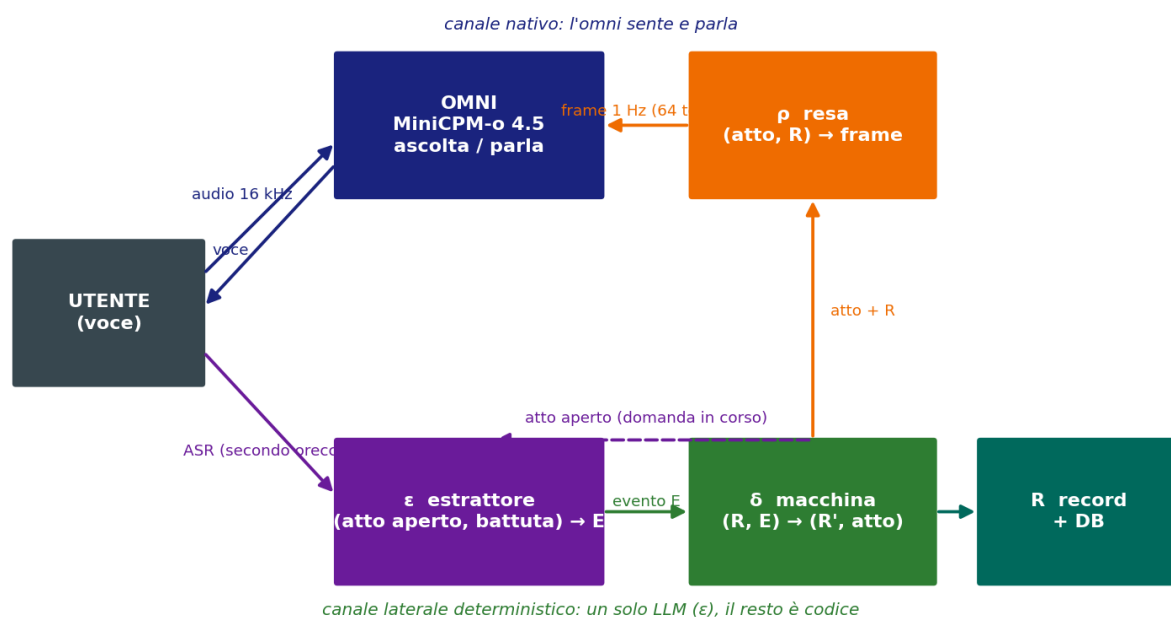


Figura 1 · Il ciclo: l'utente è sentito due volte (canale nativo dell'omni; secondo orecchio ASR → ε); la macchina δ produce un atto e un record; la resa p li disegna nel frame che l'omni legge.

2.1 Gli oggetti

R, il record. Uno schema tipato di slot con validatori. L'**intento è uno slot** come gli altri (verifica / prenotazione / ...): non esistono funzioni per dominio. Il compito è finito quando gli slot richiesti dall'intento sono validi e l'azione è stata eseguita.

E, gli eventi. Tutto ciò che un utente può fare a un record, e nient'altro:

- **Set(slot, valore)** — fornire o correggere un valore, anche fuori turno, anche più di uno per battuta

- **Answer(sì | no)** — rispondere alla domanda aperta
- **Choose(opzione)** — scegliere tra le opzioni proposte
- **Abort** — lasciar perdere
- **Nothing** — saluti, esitazioni, chiacchiere

A, gli atti. Le domande che la macchina può porre, al più una aperta alla volta: **ASK(slot)**, **OFFER(opzioni)**, **CONFIRM(record)**, **TELL(esito)**, **NONE**.

2.2 Le funzioni

ε, l'estrattore: (atto aperto, battuta) → E. Interpreta la battuta *relativamente alla domanda aperta*: dopo ASK(day) un numero è Set(day); dopo CONFIRM un "sì" è Answer(sì); in qualunque momento "vorrei prenotare" è Set(intent). È la struttura domanda/risposta della conversazione. Il modello non deve conoscere lo stato, solo la domanda; e non deve calcolare nulla.

δ, la macchina: (R, E) → (R', atto). Deterministica e **idempotente** (lo stesso Set due volte non cambia nulla): merge, validazione, esecuzione sul DB, scelta del prossimo atto. Tutto il dominio sta qui, in if/else.

ρ, la resa: (atto, R) → frame. Tabelle, mai un modello. L'omni è l'attore: rende l'atto in voce con il suo tono.

2.3 Completezza

Ogni mossa dell'utente è una risposta alla domanda aperta, una nuova assegnazione, un abbandono, o niente: l'insieme E è chiuso. Ogni compito esprimibile come **modulo più azioni** (raccogli valori, eventualmente scegli, conferma, esegui, riferisci) è una δ. Cambiare dominio significa cambiare R e δ, non E, non A, non ε. Cambiare lingua significa cambiare le parole sul frame. È la tesi dei sistemi a frame (slot-filling con conferma), la stessa di Rasa: collect, action, branch.

2.4 Dove non è completo

- **Larghezza del canale:** 64 token, 448 pixel, tre righe. OFFER regge 2-3 opzioni; liste, testi, condizioni non entrano: vanno ridotti dalla macchina o non esserci.
- **Un atto per frame:** un solo compito in fuoco. Richieste intrecciate vanno serializzate da δ.
- **Niente calcolo dal lato dell'attore:** aritmetica e ricerche sono di δ, e finiscono nello stato. È un limite voluto.
- **Canale a senso unico:** la macchina non sente l'attore, quindi non sa se l'atto è stato eseguito; lo deduce dalla battuta successiva. Tollerante, non garantito.
- **Niente "aspetta":** nessun atto esprime attesa; misurato che produce silenzi. Le attese lunghe si rendono come stato (TELL in corso).

3. Il linguaggio dell'HUD

Un frame è la quadrupla (ATTO, OGGETTO, STATO, TICK): sopra l'atto (verbo + oggetto, due o tre parole, imperativo, mai "aspetta"); sotto lo stato (il record: pieno, mancante, esito); il tick è il cambio di tonalità dello sfondo che segna ogni evento, perché il canale non ha un "invia", ha solo un "mostra".



Figura 2 · Quattro frame del linguaggio: raccolta, offerta, conferma, esito. Le parole sono generate da tabelle nel codice.

Invariante	Perché
Un atto per frame	l'omni legge un'immagine alla volta; due domande = nessuna
Oggetto di ≤ 3 parole, maiuscolo	leggibilità a 448 px; frasi lunghe vengono lette ad alta voce o ignorate
Verbi imperativi, mai "wait"	misurato: "please wait" → silenzi di 30 s, "one moment" → attrattore dell'attesa
Stato sempre presente, anche con atto NONE	è la memoria del compito, esterna al modello e alla sua finestra di contesto
Ogni cambio di atto o stato è un tick	un valore respinto o una conferma ripetuta devono produrre pixel nuovi
Parole da tabelle, mai da un modello	coerenza tra ciò che l'HUD segna e ciò che la macchina fa (regola di Alessandro)

4. Il sistema attuale, mappato sull'algebra

La forma è quella dell'algebra, con due compressioni pragmatiche fatte per il modello locale da 1,7B.

Oggetto algebrico	Nel sistema attuale	Scostamento
R, record	slot month / day / time / intent, validatori nel gateway (mese = nome, giorno 1-31, ora HH:MM), DB in data/hud_db.json	—
E, eventi	tre strumenti fissi book / check_availability / cancel; nessuna chiamata = Nothing	il significato cambia con lo stato ("book senza campi" = Answer(s)); la versione con accept/provide era l'algebra pulita (cloud 24/24), tolta per semplicità
A, atti	stati IDLE / COLLECTING / CONFIRM / DONE con MISSING, WAIT FOR USER CONFIRMATION, esiti	l'atto è implicito nello stato; non c'è ancora la metà "sopra" del frame
ε, estrattore	VAD nel browser → Whisper small (GPU) → gemini-3.5-flash-lite via API Cline (locale Qwen3-1.7B come fallback); riga di stato senza valori; ultime 6 righe del dialogo solo sul cloud	gira a fine turno (VAD), non in streaming: corsa con la risposta dell'omni e frasi spezzate
δ, macchina	_hud_fsm_apply: merge idempotente, campi indipendenti dall'ordine, numero secco = risposta alla domanda corrente, conferma prima di scrivere	—
ρ, resa	themeFor(): tabella stato → frame; impronta = pixel; sfondo alternato a ogni seq	—
Misure	regressioni: cloud 51/55 base e 24/24 dialogo (~1 s); locale 47-52/55 e 17/20 (0,5-0,8 s); dal vivo: 6,5 min coerenti con trp 1,0 + finestra KV	

5. Cosa cambierebbe davvero

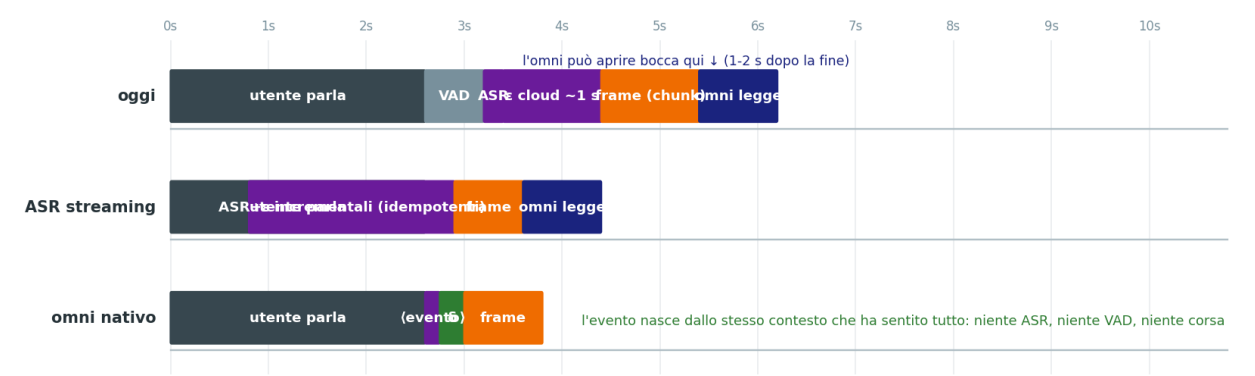


Figura 3 · Linea del tempo dalla fine della frase: oggi il frame arriva ~1,8-2,8 s dopo (l'omni può già aver risposto); con ASR in streaming arriva mentre l'utente finisce; con l'omni che emette gli eventi non c'è più il secondo orecchio.

5.1 Senza toccare l'omni

Intervento	Cosa risolve	Costo	Impatto
ASR in streaming + eventi incrementali: la trascrizione cresce, ε gira ogni ~1 s sul testo parziale, δ idempotente assorbe i duplicati; il frame è pronto quando l'utente smette	la corsa tra frame e risposta dell'omni; il VAD che spezza le frasi (i pezzi diventano Set successivi); il turno perso quando l'estrattore è occupato	un ASR streaming (Whisper con finestra scorrevole o modello streaming), stessa ε, stessa δ	grande: è il collo di bottiglia misurato (1,8-2,8 s fine frase → frame)

Intervento	Cosa risolve	Costo	Impatto
Decodifica vincolata per il locale: grammatica JSON imposta ai token, solo le chiamate valide per l'atto aperto	formato rotto, campi spazzatura (day='of'), scelta di 'none' su richieste vere	basso (logits processor / outlines)	alto sul fallback: da 47 verso 52-55/55
Contratto pulito: Set / Answer / Choose / Abort con l'intento come slot; strumenti filtrati per atto aperto	semantica incoerente degli strumenti attuali; lo spazio di uscita per stato diventa minimo	basso (già fatto una volta: cloud 20/20)	medio: robustezza e generalizzazione a nuovi domini senza toccare €
Frame a due metà: sopra l'atto (ASK: DAY, SAY: BOOKED), sotto lo stato	l'omni che davanti a MISSING dice "I'll check" invece di chiedere; reazioni non uniformi	basso (tabella stato → atto)	medio; da misurare a variabile singola
Imbuto con conteggi: δ interroga il DB prima di chiedere ("12 giorni liberi ad aprile, manca il giorno"; 2-3 opzioni → OFFER)	l'operatore che chiede a vuoto; il "prossimo libero"	medio (ricerche nel DB, già scritte una volta)	medio, di prodotto
VAD meno aggressivo (600 → 800-900 ms)	frasi spezzate	nullo	piccolo, subito

5.2 Toccando l'omni

L'omni che emette gli eventi da solo. Nel suo flusso di testo, token non parlati del tipo (set day=20), (answer yes), (abort), ignorati dal decoder vocale, letti dalla macchina δ. Sparirebbero il VAD, l'ASR, l'estrattore, la corsa tra frame e risposta, le frasi spezzate, le anafore: l'evento nascerebbe dallo stesso contesto che ha sentito tutto, con la latenza di un token. L'architettura lo consente: i token di controllo non parlati esistono già (listen/speak, fine chunk, fine turno) e la separazione testo → voce passa da un decoder vocale separato.

Cosa serve. Dati: unità omni-flow con l'annotazione degli eventi al posto giusto nel flusso (poche centinaia di dialoghi di sportello bastano per il formato; la generalità viene dal modello). Training: stadio-ruolo sulla LoRA del backbone, come per l'italiano, con il blocco visivo presente (il frame resta l'ingresso). Gate: la stessa regressione di oggi, letta dal flusso di testo invece che dall'estrattore, più il test dal vivo.

Rischi. Il testo dell'omni è legato al parlato: va insegnato che i token evento non si dicono; il rischio è che li pronunci o che li ometta. Il turn-taking può risentirne (ogni token in più nel turno ritarda il parlato di ~0,1 s). Le sessioni lunghe restano da verificare (finestra KV). È una scommessa di ricerca, ma è l'unica che rimuove il secondo orecchio invece di renderlo più veloce.

Cosa sopravvive al salto. Tutto il lato macchina e HUD: R, δ, ρ, il linguaggio del frame, le regressioni. Cambia solo chi produce E.

6. Percorso consigliato

- **Ora, senza toccare l'omni:** ASR in streaming con eventi incrementali (il collo di bottiglia misurato), poi decodifica vincolata per il locale, poi il contratto pulito e il frame a due metà come esperimenti a variabile singola, misurati con le regressioni e le righe STATO delle sessioni.
- **In parallelo, sui dati:** raccogliere i dialoghi di sportello (già registrati in data/sessions) e annotare gli eventi: sono il set di training per l'omni nativo e, intanto, la regressione dal vivo.
- **Poi, toccando l'omni:** stadio-ruolo con i token evento, gate in doppio contesto (training e deploy), giudice il test dal vivo. Se passa, € diventa interno e il sistema resta identico per il resto.

- **Cosa non fare:** strumenti di fuga per il modello (wait, none quando è attraente), istruzioni discorsive sul frame, un secondo turn-taking sopra quello nativo, valori copiabili nello stato dove non sono azionabili. Tutte cose già misurate.

Fonti interne: plan/ramo-hud.md (specifica, test dal vivo 1-9, misure), analisi-toolcalling.md (§1-8: errori di copia, prompt, cloud, operazioni, semplificazione), tools/tool_agent_eval*.py (75 casi). Esterne: doc Qwen function calling; Rasa LLM command generators; benchmark tool-calling dei modelli piccoli; confronto on-device 2026 (BFCL).